30-Day FreeRTOS Course for ESP32 Using ESP-IDF

(Day 19)

“Memory Management in FreeRTOS”

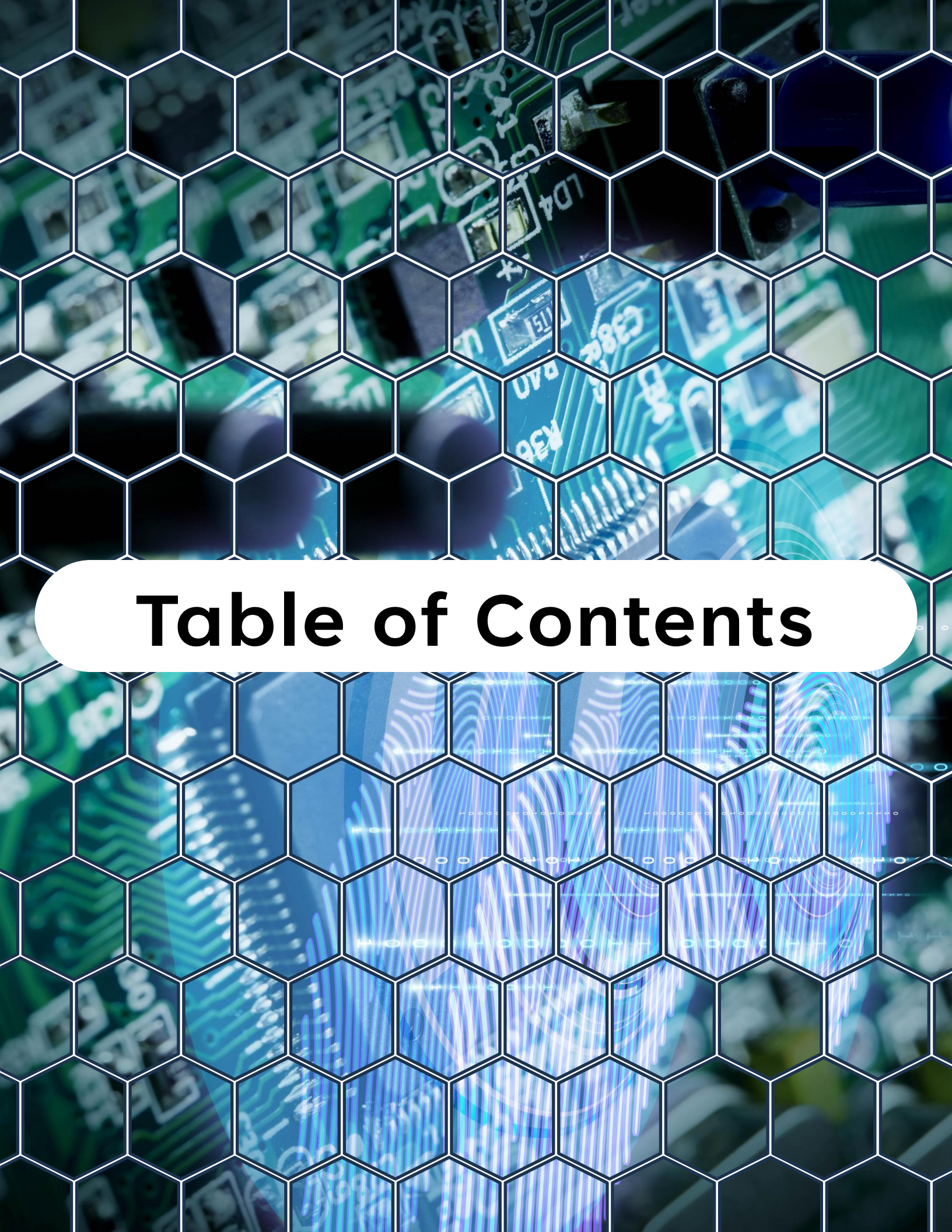
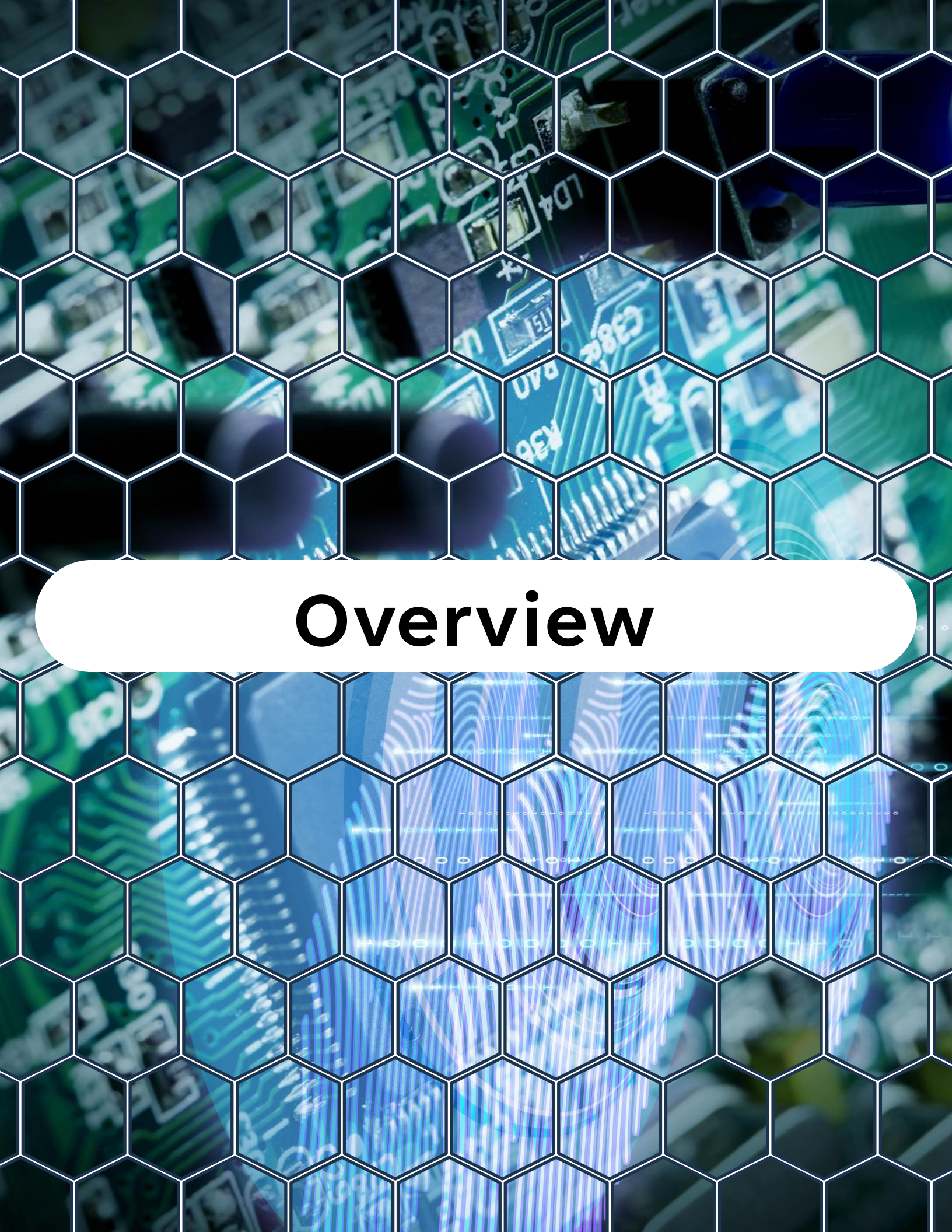


Table of Contents

Table of Contents



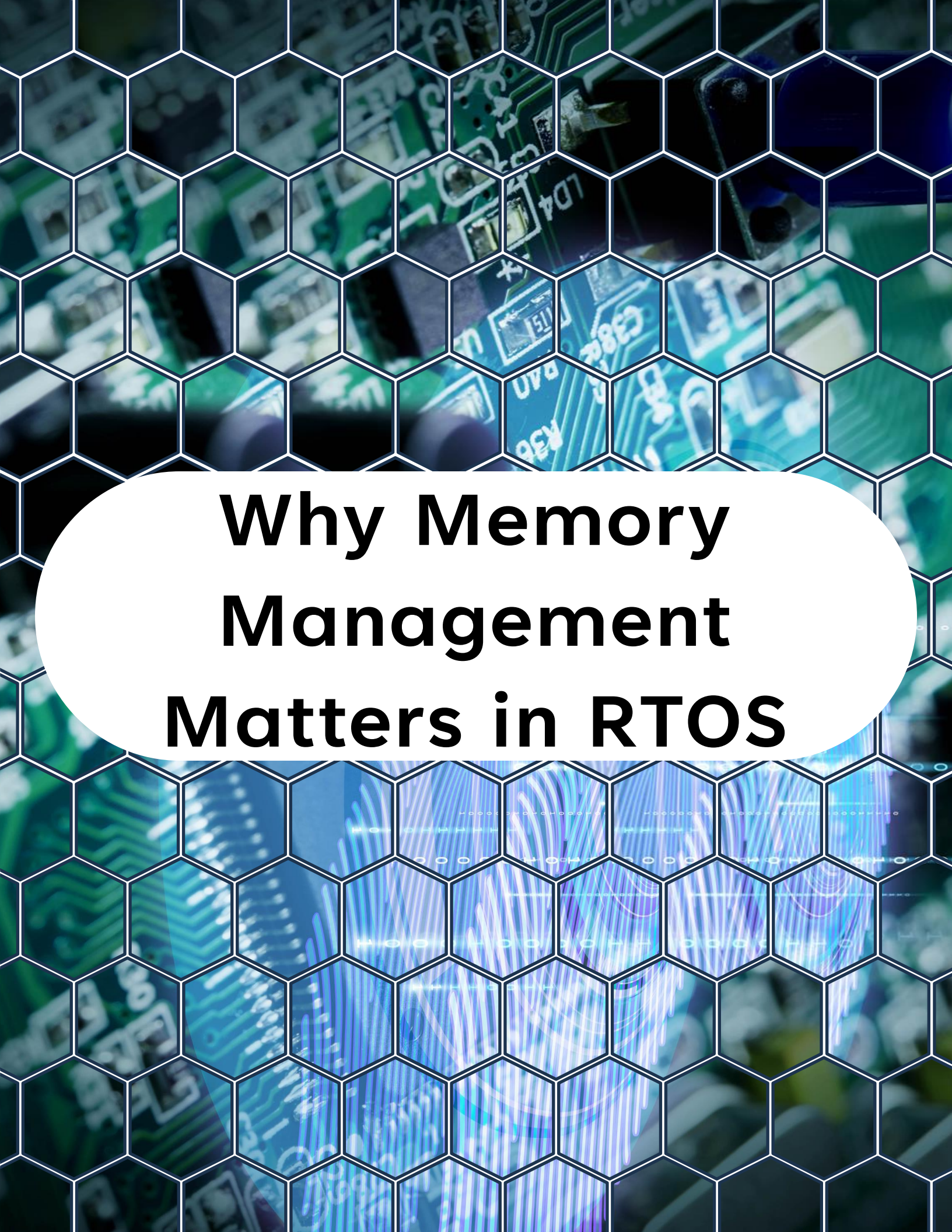
Overview

Overview

On **Day 19**, you'll learn:

- How FreeRTOS manages memory on embedded systems
- The role of the **heap** in task stacks, queues, semaphores, and timers
- The different **heap memory allocation schemes (heap_1 to heap_5)**
- How to configure FreeRTOS memory management in ESP-IDF
- How to use **static allocation** instead of dynamic allocation
- Best practices for efficient and safe memory use on ESP32

By the end of this lesson, you'll understand how to **allocate, manage, and optimize memory** in FreeRTOS-based ESP32 applications.



Why Memory Management Matters in RTOS

Why Memory Management Matters in RTOS

Unlike PCs, embedded systems like ESP32 have **limited RAM** (e.g., 320 KB internal RAM + optional external PSRAM).

Poor memory management can lead to:

- **Heap fragmentation**
- **Task creation failures**
- **Unpredictable system crashes**

📌 FreeRTOS provides flexible heap allocation schemes so you can **balance performance and safety for your application**.

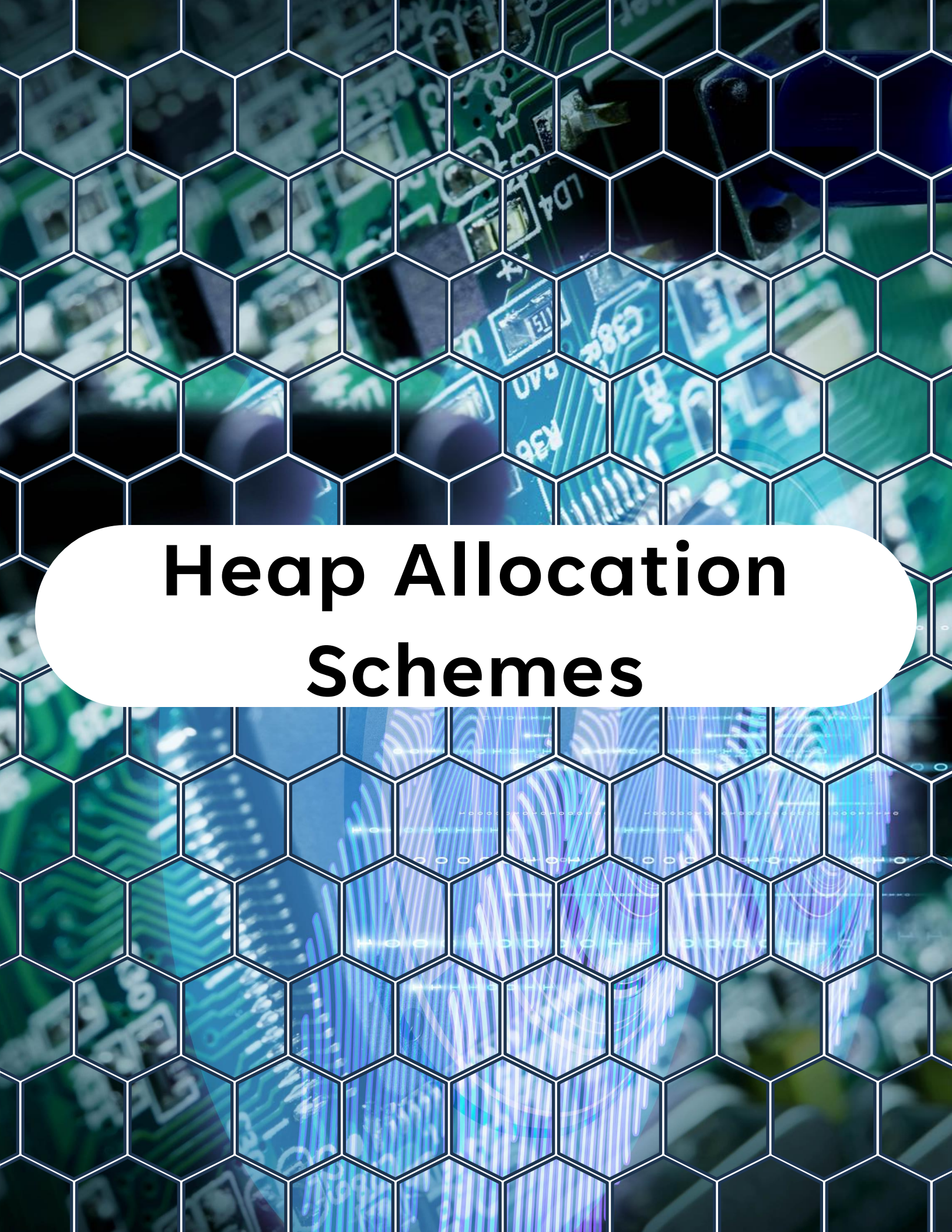
The Heap in FreeRTOS

The Heap in FreeRTOS

The heap is a global memory pool used for:

- Creating tasks (**xTaskCreate**)
- Creating queues, semaphores, and event groups
- Creating software timers

FreeRTOS does not use the standard `malloc()/free()` from libc. Instead, it provides its own heap allocators, selected at compile time.



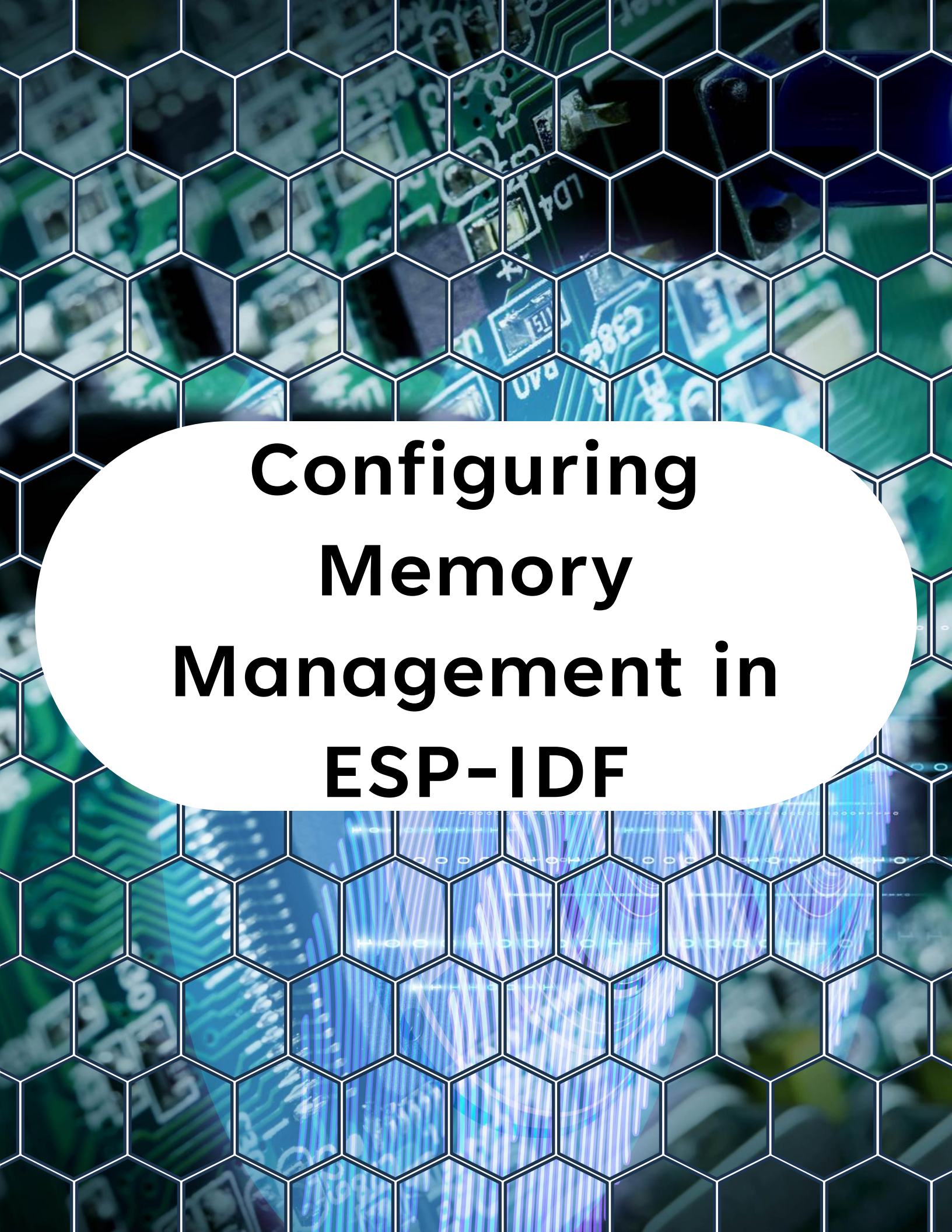
Heap Allocation Schemes

Heap Allocation Schemes

FreeRTOS offers five heap implementations in `heap_x.c`:

Scheme	Description
heap_1	Simplest, only malloc(). No free(). No fragmentation.
heap_2	Allows free(). Simple algorithms but can fragment.
heap_3	Uses standard C malloc()/free(). Depends on system allocator.
heap_4	Best balance. Allows free(), uses coalescing to reduce fragmentation.
heap_5	Like heap_4, but supports multiple memory regions (e.g., internal + external RAM).

📌 ESP-IDF typically uses heap_4 by default, as it handles fragmentation well.


Configuring
Memory
Management in
ESP-IDF

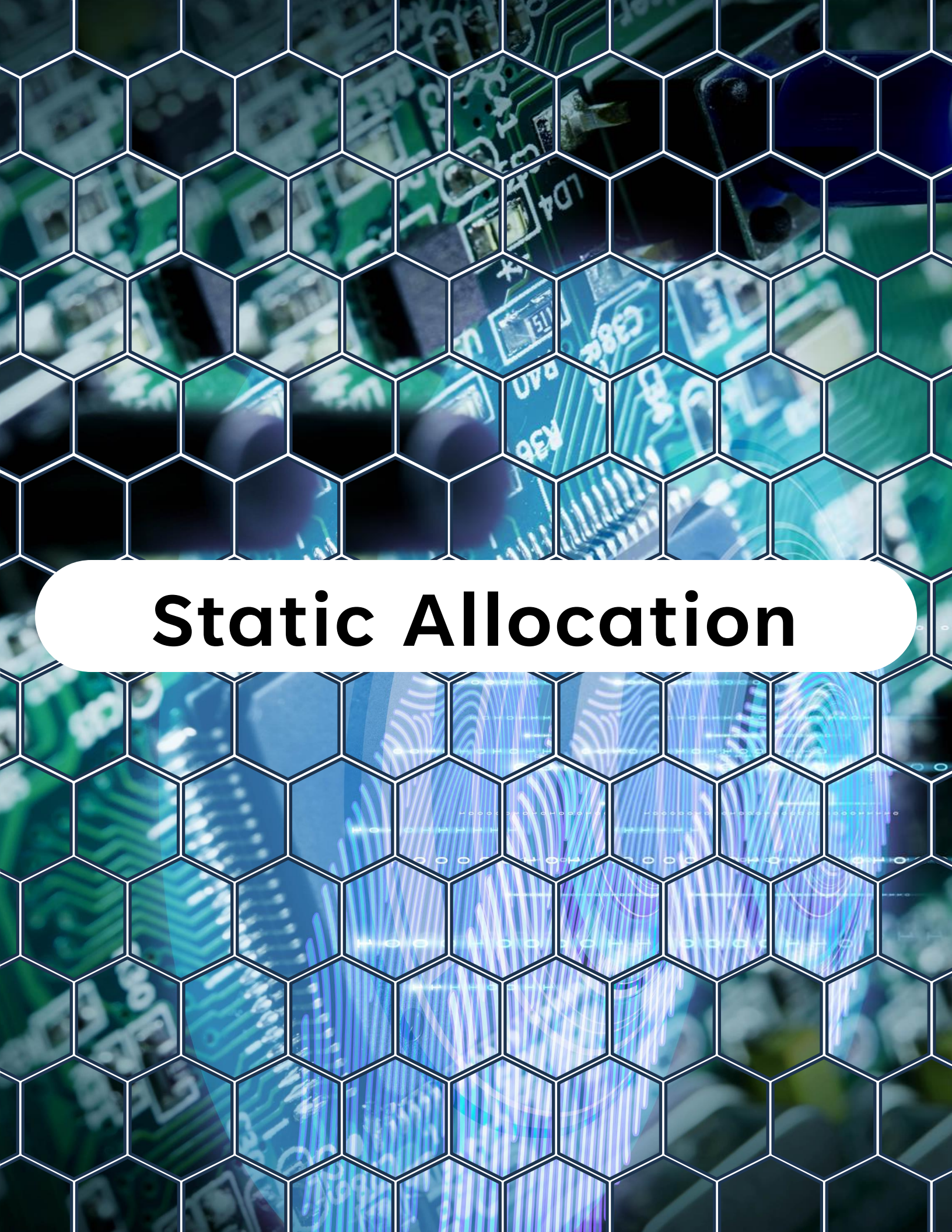
Configuring Memory Management in ESP-IDF

In ESP-IDF, memory allocation settings are controlled via:

`idf.py menuconfig → Component config → FreeRTOS → Heap memory`

You can:

- Select which `heap_x.c` implementation to use.
- Enable **static allocation support**.
- Configure **stack overflow checking**.
- Enable **heap poisoning** for debugging memory corruption.
- Force the entire heap component to be **placed in flash memory**



Static Allocation

Static Allocation

Instead of using dynamic allocation (`xTaskCreate`), FreeRTOS also supports static allocation (`xTaskCreateStatic`).

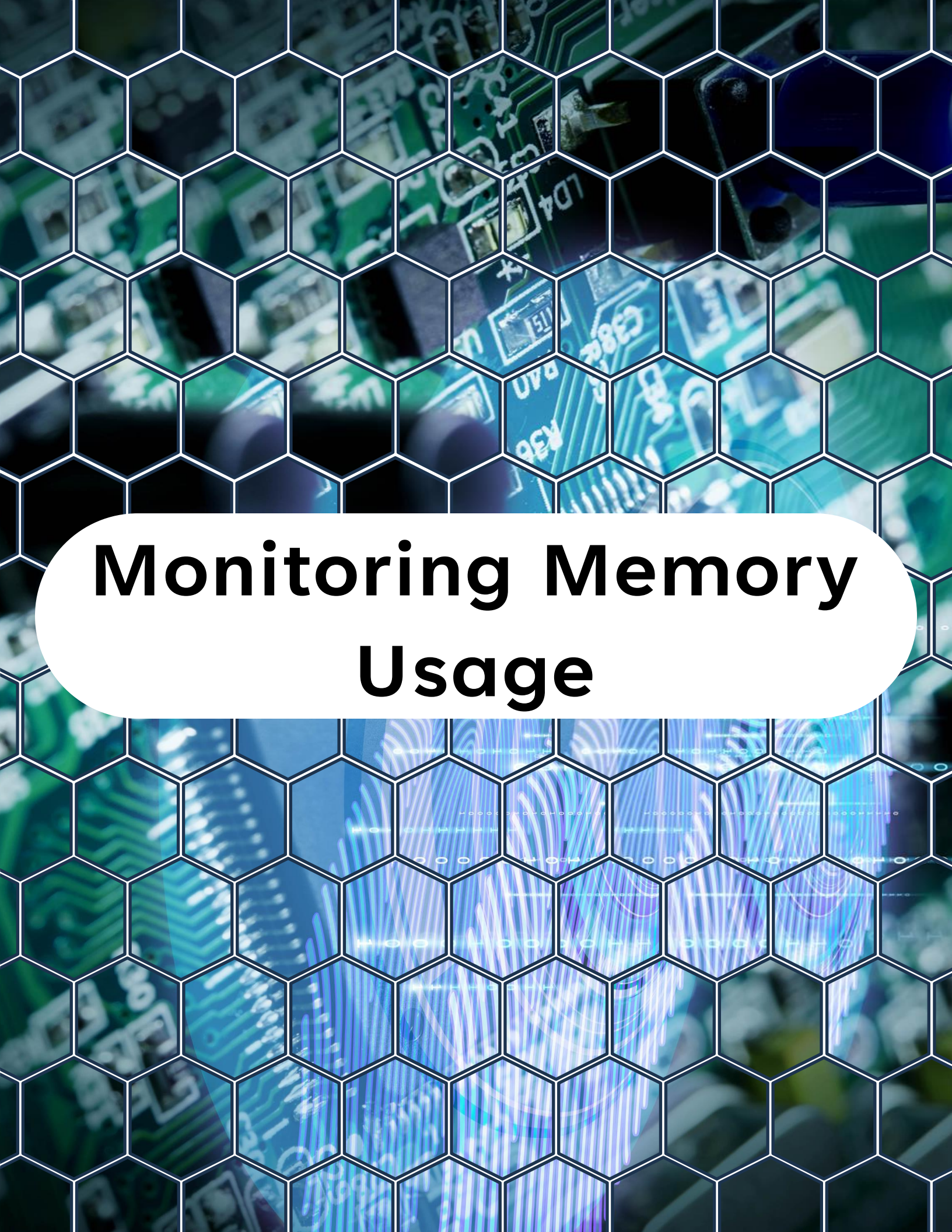
Example – Statically Allocated Task

```c
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "esp_log.h"

#define TAG "static-demo"

// Start around 2-3 KB for simple tasks that call library
// code or printf(), then tune using
// uxTaskGetStackHighWaterMark2()
#define STACK_SIZE_BYTES (3 * 1024)

// Task control block (TCB) for the task
StaticTask_t task_buffer;

// Stack memory for the task
StackType_t task_stack[STACK_SIZE_BYTES];

// Task function
void static_task(void *pvParameters) {
    while (1) {
        ESP_LOGI(TAG, "Running static task; free stack (bytes): %u",
                 (unsigned)uxTaskGetStackHighWaterMark2(NULL));
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}

// Main application entry point
void app_main() {
```

Static Allocation

```
27 // Main application entry point
28 void app_main() {
29     // Create task without using the heap
30     TaskHandle_t handle = xTaskCreateStatic(
31         static_task,      // Task function
32         "StaticTask",     // Task name
33         STACK_SIZE_BYTES,       // Stack size
34         NULL,             // Parameters
35         5,                // Priority
36         task_stack,       // Stack memory
37         &task_buffer      // Task control block
38     );
39 
40     if (handle == NULL) {
41         ESP_LOGE(TAG,
42             "xTaskCreateStatic failed (insufficient stack/invalid args)");
43     }
44 }
```

No heap allocation → predictable, safer for memory-constrained applications.



Monitoring Memory Usage

Monitoring Memory Usage

FreeRTOS provides APIs to check memory usage:

- `xPortGetFreeHeapSize()` → total free heap.
- `xPortGetMinimumEverFreeHeapSize()` → lowest heap ever recorded (good for tuning).

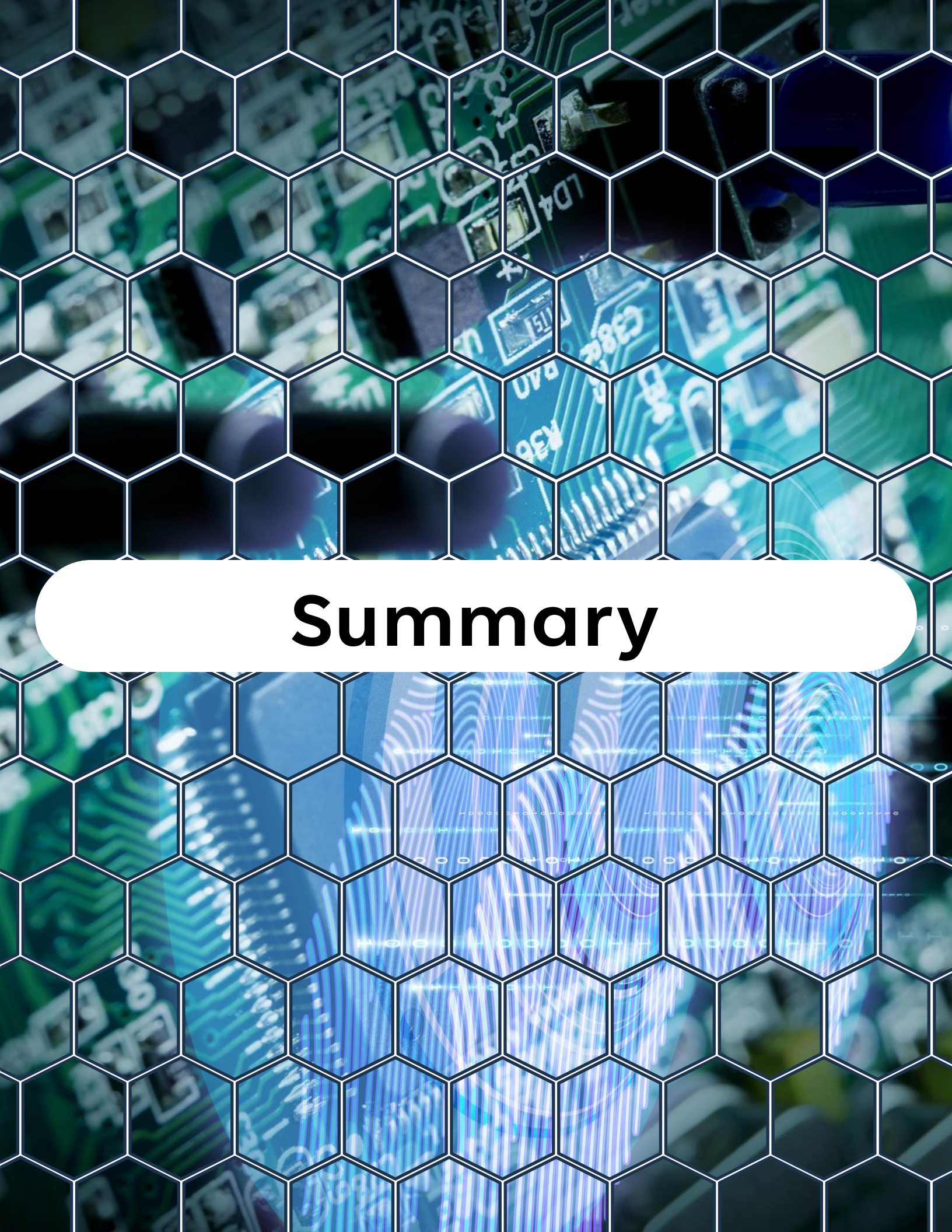
ESP-IDF also includes:

- `heap_caps_get_free_size(MALLOC_CAP_8BIT)` → free memory of a given capability.
- `heap_caps_print_heap_info(MALLOC_CAP_8BIT)` → detailed heap usage.

Best Practices

Best Practices

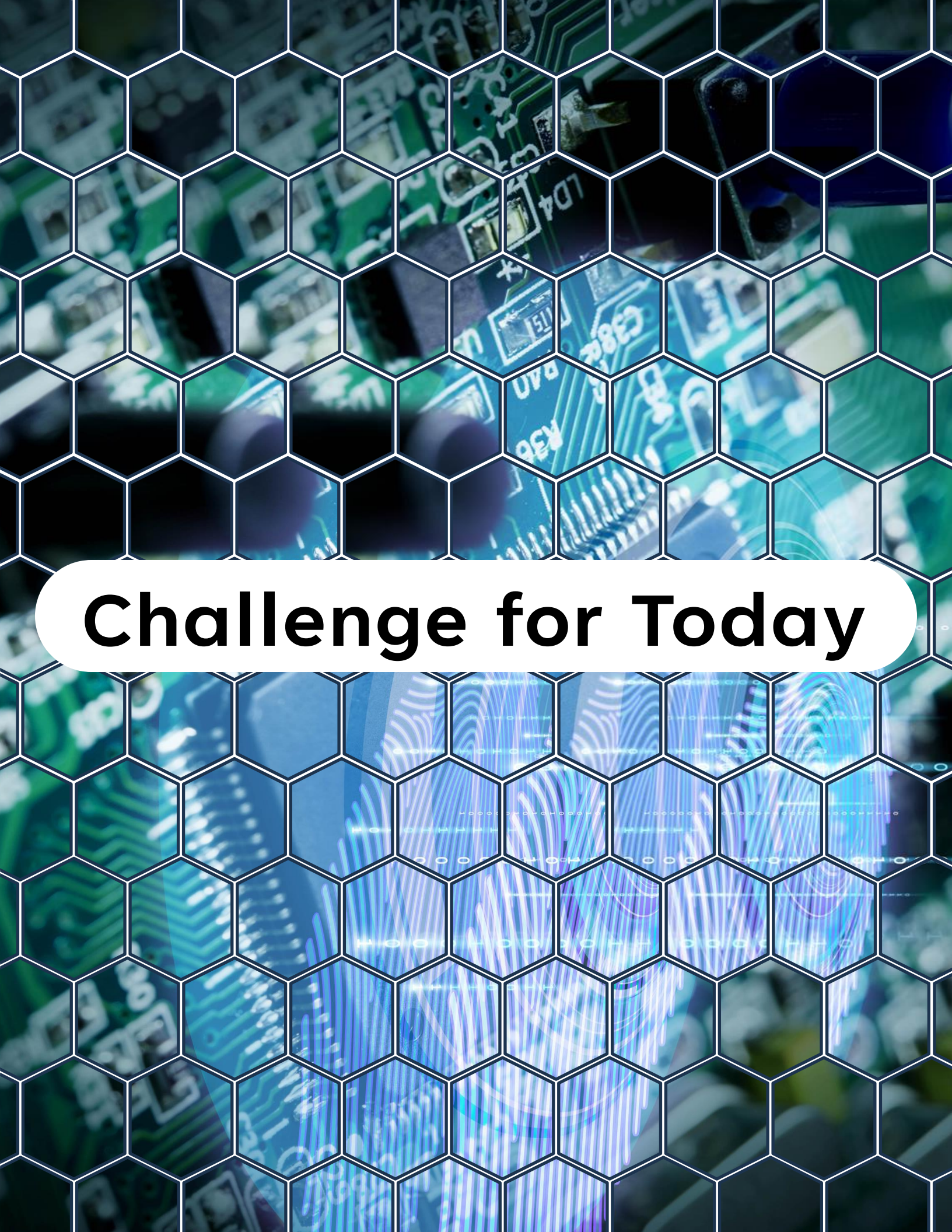
- ✅ Use **heap_4** (or **heap_5** if using multiple regions).
- ✅ Prefer **static allocation** for critical tasks (predictable).
- ✅ Regularly check `xPortGetMinimumEverFreeHeapSize()` to ensure enough headroom.
- ✅ Keep task stacks small and measured — use `uxTaskGetStackHighWaterMark()` to tune.
- ✅ For heavy data, use `external PSRAM` (on ESP32-WROVER or S3 with PSRAM).
- ✅ Enable **stack overflow detection** in menuconfig.



Summary

Summary

- FreeRTOS memory is managed through **heap_x.c** implementations.
- ESP-IDF defaults to **heap_4**, which is safe and efficient.
- Use **static allocation** when predictability is required.
- Monitor heap and stack usage to prevent crashes.
- Use PSRAM or heap_5 if your app requires multiple memory regions.


Challenge for Today

Challenge for Today

🛠️ Create two tasks:

- One using xTaskCreate (dynamic).
- One using xTaskCreateStatic (static).
- Print memory statistics using xPortGetFreeHeapSize() before and after creation.
- Compare the difference and discuss which approach is safer for real-time systems.

"Thanks for watching!

If you enjoyed this video,

make sure to hit the like button and subscribe to stay updated with my latest content.

Don't forget to check out my other videos for more tips and tutorials on Embedded C, Python, hardware designs, etc. Keep exploring, keep learning, and I'll see you in the next video!"

DISCOVER, LEARN, SUCCEED!

https://www.linkedin.com/in/yamil-garcia
https://www.youtube.com/@LearningByTutorials
https://github.com/god233012yamil